

PHOTO MEMORIES AI - BACKEND ENGINEER PRD V2

Manual Event Creation + Photographer Dashboard Model

Project Owner: Krish

Your Role: Backend Engineer (Node.js + Express)

Partner Role: Frontend Engineer (Next.js)

Build Timeline: 6-8 weeks

Hosting: Railway/Render (separate from frontend)

0. UPDATED WORKFLOW (READ THIS FIRST)

Old Workflow (Abandoned)

Photographer submits → AI auto-generates landing
page → Done

New Workflow (Current)

1. Photographer creates REQUEST (paid)
 - Fills form: event name, date, couple, theme, plan
 - Selects subscription (Base/Medium/Pro)
 - Pays ₹X
 - Status: "pending"
2. Krish (Admin) Creates Pages
 - Designs invitation card (manual/Figma)
 - Designs landing page (manual/Figma)
 - Creates gallery structure
 - Marks as "ready_for_upload"
 - Photographer notified
3. Photographer Uploads Photos (After Event)
 - Goes to their album
 - Uploads all photos/videos
 - Can organize, delete, edit
 - Status: "live"
4. Guest Accesses Album
 - Gets shareable link
 - Sees invitation card
 - Views gallery (based on plan)
 - Downloads/shares

1. YOUR BACKEND RESPONSIBILITIES

✓ YOU BUILD

- Authentication (signup, login, JWT)
- Photographer request submission API
- Admin panel backend (mark ready, view requests)
- Photo upload endpoint
- Gallery generation (after photos uploaded)
- Subscription/payment integration
- Email notifications
- Analytics tracking
- Subdomain management
- Security/Encryption
- Database queries
- Rate limiting

✗ YOU DON'T BUILD

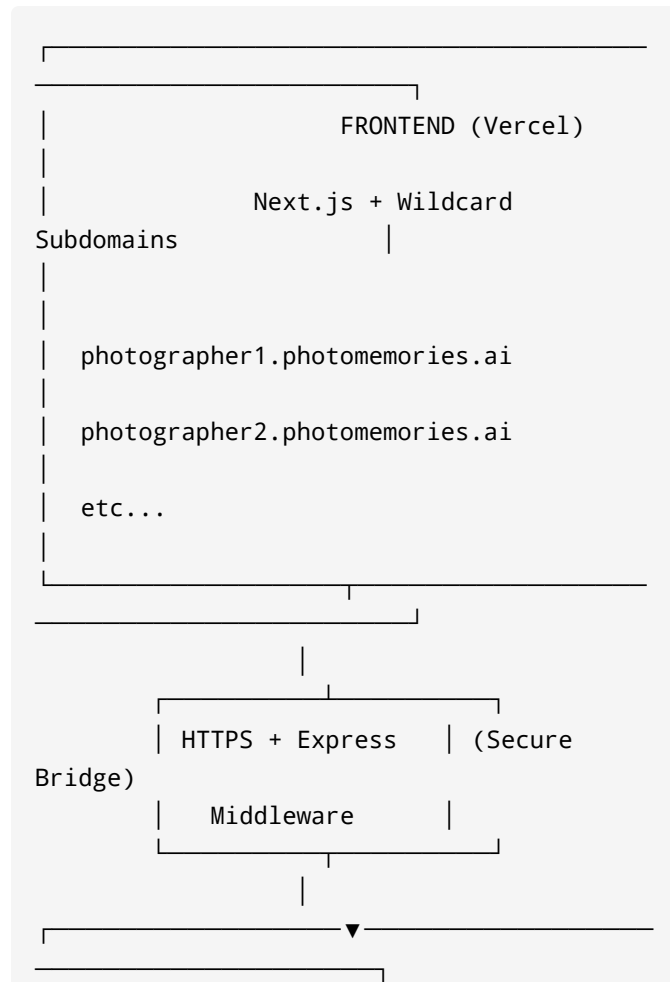
- Frontend UI
- React components
- Photo compression logic (Cloudinary handles)
- Email sending (uses Nodemailer)
- Figma design (Krish does manual design)

🔗 INTEGRATION POINTS

- Frontend calls your APIs
- You generate gallery data, frontend displays it

- You track analytics, frontend shows charts
- You handle subdomain routing logic, frontend consumed it

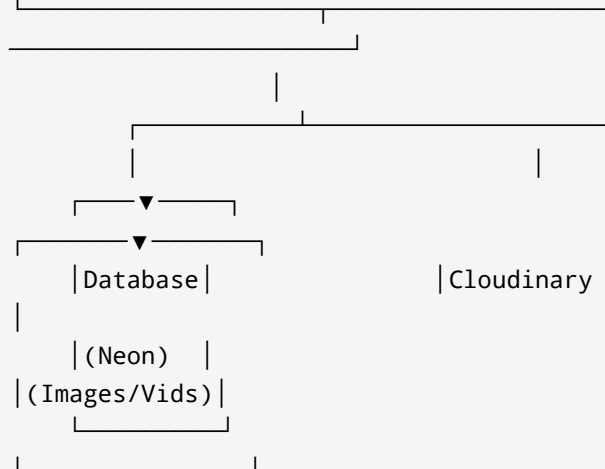
2. ARCHITECTURE OVERVIEW



BACKEND (Railway/Render)

Node.js + Express + Security

- API Routes
- Database Logic (PostgreSQL)
- Auth & Encryption
- Payment Processing
- File Upload Handling
- Email Service



Key: Frontend and Backend on different servers.
Express middleware handles secure communication.

3. DATABASE SCHEMA

Users Table

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  name VARCHAR(255),  
  role ENUM('photographer', 'admin')  
  DEFAULT 'photographer',  
  company_name VARCHAR(255),  
  phone VARCHAR(20),  
  subdomain VARCHAR(100) UNIQUE,  
  city VARCHAR(100),  
  created_at TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP,  
  last_login TIMESTAMP,  
  is_active BOOLEAN DEFAULT true,  
  email_verified BOOLEAN DEFAULT false,  
  verification_token VARCHAR(255),  
  password_reset_token VARCHAR(255),  
  password_reset_expires TIMESTAMP  
);  
CREATE INDEX idx_users_email ON  
users(email);
```

```
CREATE INDEX idx_users_subdomain ON  
users(subdomain);
```

NEW: `subdomain` field - each photographer gets
unique subdomain

Events Table

```
CREATE TABLE events (  
  id SERIAL PRIMARY KEY,  
  photographer_id INTEGER NOT NULL  
REFERENCES users(id) ON DELETE CASCADE,  
  event_name VARCHAR(255) NOT NULL,  
  couple_names VARCHAR(255),  
  event_date DATE NOT NULL,  
  theme VARCHAR(50) NOT NULL,  
  location VARCHAR(255),  
  client_email VARCHAR(255),  
  client_phone VARCHAR(20),  
  plan ENUM('base', 'medium', 'pro')  
DEFAULT 'base',  
  status ENUM('pending',  
'ready_for_upload', 'live', 'archived')  
DEFAULT 'pending',  
  unique_slug VARCHAR(255) UNIQUE NOT NULL,  
  description TEXT,  
  
  -- Admin-created content (manual design)  
  invitation_html TEXT,  
  landing_page_html TEXT,  
  
  -- Photographer data
```

```

photo_count INTEGER DEFAULT 0,
video_count INTEGER DEFAULT 0,
view_count INTEGER DEFAULT 0,

-- Subscription info
amount_paid DECIMAL(10, 2),
payment_status ENUM('pending',
'completed', 'failed') DEFAULT 'pending',

-- Tracking
created_at TIMESTAMP DEFAULT
CURRENT_TIMESTAMP,
ready_at TIMESTAMP,
went_live_at TIMESTAMP,

-- Admin notes
admin_notes TEXT
);
CREATE INDEX idx_events_photographer ON
events(photographer_id);
CREATE INDEX idx_events_slug ON
events(unique_slug);
CREATE INDEX idx_events_status ON
events(status);

```

NEW:

- `plan` field (which subscription)
- `status: 'ready_for_upload'` (new state after we create)
- `invitation_html` (you store HTML we created)

- `landing_page_html` (you store HTML we created)
- `amount_paid` (they paid us)

Photos Table

```
CREATE TABLE photos (  
  id SERIAL PRIMARY KEY,  
  event_id INTEGER NOT NULL REFERENCES  
events(id) ON DELETE CASCADE,  
  uploaded_by INTEGER REFERENCES users(id),  
  clouinary_id VARCHAR(255) NOT NULL,  
  clouinary_url VARCHAR(500) NOT NULL,  
  clouinary_thumb_url VARCHAR(500),  
  file_type ENUM('photo', 'video') DEFAULT  
'photo',  
  upload_date TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP,  
  created_at TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);  
CREATE INDEX idx_photos_event ON  
photos(event_id);  
CREATE INDEX idx_photos_uploaded_by ON  
photos(uploaded_by);
```

NEW:

- `uploaded_by` (track which photographer uploaded)
- `file_type` (photo or video)

- Can be added AFTER event goes "ready_for_upload"

Subscriptions Table (New)

```
CREATE TABLE subscriptions (  
  id SERIAL PRIMARY KEY,  
  photographer_id INTEGER NOT NULL  
  REFERENCES users(id),  
  event_id INTEGER NOT NULL REFERENCES  
  events(id),  
  plan ENUM('base', 'medium', 'pro'),  
  amount DECIMAL(10, 2),  
  features JSONB,  
  purchased_at TIMESTAMP DEFAULT  
  CURRENT_TIMESTAMP,  
  expires_at TIMESTAMP  
);
```

Analytics Table

```
CREATE TABLE analytics (  
  id SERIAL PRIMARY KEY,  
  event_id INTEGER NOT NULL REFERENCES  
  events(id) ON DELETE CASCADE,  
  action VARCHAR(50) NOT NULL,  
  action_date DATE NOT NULL,  
  user_ip VARCHAR(45),  
  user_agent VARCHAR(500),  
  referrer VARCHAR(500),  
  created_at TIMESTAMP DEFAULT
```

```
CURRENT_TIMESTAMP  
);  
Create INDEX idx_analytics_event ON  
analytics(event_id);
```

AdminLogs Table

```
CREATE TABLE admin_logs (  
  id SERIAL PRIMARY KEY,  
  admin_id INTEGER NOT NULL REFERENCES  
users(id),  
  action VARCHAR(100) NOT NULL,  
  entity_type VARCHAR(50),  
  entity_id INTEGER,  
  reason TEXT,  
  created_at TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
);
```

4. API ENDPOINTS (YOUR RESPONSIBILITY)

Authentication

```
POST /api/auth/signup  
  Request: { email, password, name,  
company_name, phone }  
  Response: { success, user, token,
```

```
subdomain }  
  YOUR JOB: Hash password, create  
subdomain, send verification email  
  
POST /api/auth/login  
  Request: { email, password }  
  Response: { success, user, token }  
  
POST /api/auth/logout  
  Request: {} (protected)  
  Response: { success }  
  
POST /api/auth/verify-email  
  Request: { token }  
  Response: { success }  
  
POST /api/auth/forgot-password & reset-  
password  
  Similar to before
```

Photographer: Create Event Request

```
POST /api/events/request  
Auth: Required (Photographer)  
Request Body:  
{  
  "event_name": "Aarav & Priya Wedding",  
  "couple_names": "Aarav & Priya",  
  "event_date": "2024-12-15",  
  "theme": "traditional",  
  "location": "Delhi",  
  "client_email": "client@example.com",
```

```
"plan": "pro",  
"amount": 25000  
}
```

YOUR JOB:

- Validate input
- Verify plan amount
- Create Razorpay/Stripe payment intent
- Create event with status: "pending"
- Return: { event_id, payment_url }

Response (201):

```
{  
  "success": true,  
  "event_id": 42,  
  "payment_url":  
  "https://checkout.razorpay.com/...",  
  "plan": "pro"  
}
```

Payment Webhook

POST /api/payments/webhook

Request: From Razorpay/Stripe

YOUR JOB:

- Verify payment signature
- Update event: payment_status =
"completed"
- Send email to photographer: "Admin will
create your pages"
- Send email to admin (Krish): "New
request from photographer X"

```
Response: { success }
```

Photographer: View Their Events

```
GET /api/photographer/events
```

```
Auth: Required (Photographer)
```

```
Query: { page, status, sortBy }
```

YOUR JOB:

- Fetch events where photographer_id = current user
- Filter by status (pending, ready_for_upload, live)
- Return only THEIR events

Response (200):

```
{
  "success": true,
  "events": [
    {
      "id": 42,
      "event_name": "Aarav & Priya
Wedding",
      "status": "ready_for_upload",
      "plan": "pro",
      "photo_count": 0,
      "ready_at": "2024-09-25T10:30:00Z"
    }
  ]
}
```

Photographer: Get Single Event

```
GET /api/photographer/events/:eventId
Auth: Required (Photographer)
```

YOUR JOB:

- Verify photographer owns this event
- Return event details + html pages + photos

Response:

```
{
  "success": true,
  "event": {
    "id": 42,
    "event_name": "...",
    "status": "live",
    "plan": "pro",
    "invitation_html": "<html>...
</html>",
    "landing_page_html": "<html>...
</html>",
    "unique_slug": "aarav-priya-2024",
    "photos": [...]
  }
}
```

Photographer: Upload Photos

```
POST /api/events/:eventId/upload-photos
Auth: Required (Photographer)
Request: multipart/form-data { photos:
```

```
[File...], videos: [File...] }
```

CONDITIONS:

- Event status must be "ready_for_upload" or "live"
- Only photographer who owns event can upload

YOUR JOB:

- Validate files (size, type)
- Upload to Cloudinary
- Store metadata in photos table
- Update photo_count in events
- Auto-generate gallery (after first upload, set status to "live")

Response (201):

```
{  
  "success": true,  
  "uploaded_count": 5,  
  "event_status": "live",  
  "event_url":  
  "photographer1.photomemories.ai/event/aarav-  
  priya-2024"  
}
```

Photographer: Manage Photos

GET /api/events/:eventId/photos

Auth: Required (Photographer)

YOUR JOB:

- Fetch photos for this event
- Verify photographer owns event
- Return photo metadata

DELETE /api/photos/:photoId
Auth: Required (Photographer)

YOUR JOB:

- Verify photographer owns event
- Delete from Clouidinary
- Delete from database
- Update photo_count

Admin: View Pending Requests

GET /api/admin/events/pending
Auth: Required (Admin only)

YOUR JOB:

- Fetch all events with status = "pending"
- Include photographer info
- Return plan details & amount paid

Response:

```
{
  "success": true,
  "pending_events": [
    {
      "id": 42,
      "event_name": "Aarav & Priya
Wedding",
```

```
        "photographer_name": "John Doe",
        "photographer_email":
"john@example.com",
        "plan": "pro",
        "amount_paid": 25000,
        "requested_at": "2024-09-
21T10:30:00Z"
    }
]
}
```

Admin: Mark Event Ready for Upload

```
POST /api/admin/events/:eventId/mark-ready
Auth: Required (Admin)
Request Body:
{
  "invitation_html": "<html>...</html>",
  "landing_page_html": "<html>...</html>"
}

YOUR JOB:
- Verify event exists & is in "pending"
- Store HTML in database
- Update status to "ready_for_upload"
- Update ready_at timestamp
- Send email to photographer: "Album
ready! Upload your photos"
- Log action in admin_logs

Response:
{
```

```
"success": true,  
"event_id": 42,  
"status": "ready_for_upload"  
}
```

Admin: View All Events

```
GET /api/admin/events  
Auth: Required (Admin)  
Query: { status, photographer_id, page }
```

YOUR JOB:

- Return all events (not just one photographer)
- Include photographer info & revenue
- Filter by status

Public: Get Event Landing Page

```
GET /api/public/event/:slug  
Auth: Not required (Public)
```

YOUR JOB:

- Find event by slug
- Fetch photographer's subdomain
- Return event data + HTML pages

Response:

```
{  
  "success": true,  
  "event": {
```

```
        "event_name": "Aarav & Priya  
Wedding",  
        "subdomain": "john-  
photography.photomemories.ai"  
    },  
    "invitation_html": "<html>...</html>",  
    "landing_page_html": "<html>...</html>"  
}
```

Public: Get Gallery

GET /api/public/event/:slug/gallery

Auth: Not required (Public)

Query: { page, limit }

CONDITIONS:

- Check event.status = "live"
- Check subscription plan (what can they access)

YOUR JOB:

- Fetch photos for event
- Filter based on plan (Base = photos only, Medium = photos+videos, Pro = all)
- Return paginated results

Response:

```
{  
  "success": true,  
  "plan": "pro",  
  "photos": [  
    {
```

```
        "id": 101,
        "cloudinary_url": "https://...",
        "cloudinary_thumb_url":
"https://...",
        "type": "photo"
    }
],
    "pagination": { "page": 1, "total": 150
}
}
```

Public: Track Analytics

```
POST /api/analytics/track
Auth: Not required (Public)
Request:
{
    "event_slug": "aarav-priya-2024",
    "action": "view" | "gallery_view" |
"download" | "share",
    "referrer": "whatsapp"
}

YOUR JOB:
- Store in analytics table
- Capture IP, user_agent
- Increment event.view_count

Response: { success: true }
```

Subdomain Management (Internal)

```
POST /api/subdomains/create
  Auth: Required (when user signs up)
  Request: { photographer_id,
subdomain_name }

YOUR JOB:
- Check if subdomain available
- Create DNS record (or use Vercel
wildcard)
- Update users.subdomain field
- Store in subdomains table

Response: { success, subdomain: "john-
photography.photomemories.ai" }
```

5. SECURITY & ENCRYPTION

Frontend-Backend Communication

```
All traffic over HTTPS (enforced)
Express middleware:
├─ CORS: Only allow frontend domain
├─ Rate limiting: 100 requests/15 mins per
IP
├─ Input validation: Sanitize all inputs
├─ CSRF protection: If using cookies
└─ Security headers: HSTS, CSP, X-Frame-
Options
```

Password Security

```
// Hash on backend only (never frontend)
const hash = await bcrypt.hash(password,
10);

// Never send plain passwords in
logs/emails
```

Encryption at Rest (Sensitive Data)

```
// Encrypt before storing in DB
const encryptData = (data: string): string
=> {
  const iv = crypto.randomBytes(16);
  const cipher =
crypto.createCipheriv('aes-256-gcm',
Buffer.from(ENCRYPTION_KEY, 'hex'), iv);
  let encrypted = cipher.update(data,
'utf8', 'hex');
  encrypted += cipher.final('hex');
  const authTag = cipher.getAuthTag();
  return
`${iv.toString('hex')}:${authTag.toString('
hex')}:${encrypted}`;
};

// Decrypt on retrieval
const decryptData = (encrypted: string):
string => {
  const parts = encrypted.split(':');
  const iv = Buffer.from(parts[0], 'hex');
```

```
    const authTag = Buffer.from(parts[1],
    'hex');
    const decipher =
    crypto.createDecipheriv('aes-256-gcm',
    Buffer.from(ENCRYPTION_KEY, 'hex'), iv);
    decipher.setAuthTag(authTag);
    let decrypted = decipher.update(parts[2],
    'hex', 'utf8');
    decrypted += decipher.final('utf8');
    return decrypted;
};

// Encrypt: client_email, client_phone,
sensitive details
```

JWT Tokens

- Stored in httpOnly cookie (backend sets on login)
- 24-hour expiry
- Refresh token for renewal
- Invalidated on logout
- Verified on every protected request

Data Privacy

- Photographer can ONLY access their own events
- No cross-photographer data leakage
- Each subdomain isolated

- Admin can see all (that's fine, you're admin)

6. SUBDOMAIN ARCHITECTURE

How It Works

1. User signs up
POST /api/auth/signup
Backend creates subdomain: "john-photography.photomemories.ai"
Stores in users table
2. Vercel Wildcard Setup
DNS: *.photomemories.ai → Vercel
Vercel: Same Next.js app serves all subdomains
3. Backend routes subdomain
GET /api/public/event/:slug
Returns photographer's data based on subdomain header
4. Frontend receives subdomain
Renders event pages under photographer's subdomain

Database

```
CREATE TABLE subdomains (  
  id SERIAL PRIMARY KEY,  
  photographer_id INTEGER REFERENCES  
users(id),  
  subdomain_name VARCHAR(100) UNIQUE,  
  created_at TIMESTAMP,  
  is_active BOOLEAN DEFAULT true  
);
```

API for Subdomain

Photographer info available via:
GET /api/public/photographer/:subdomain
Returns: photographer name, company,
logo, etc.
Used by frontend for branding

7. DEVELOPMENT CHECKLIST

Week 1-2: Foundation

- ☐ Express setup + TypeScript
- ☐ PostgreSQL connection (Neon)
- ☐ Database schema (all tables above)
- ☐ Middleware (auth, CORS, rate limit, encryption)
- ☐ JWT authentication
- ☐ Email service (Nodemailer)

Week 3-4: Photographer APIs

- ☐ POST /api/auth/signup
- ☐ POST /api/auth/login
- ☐ POST /api/events/request
- ☐ Payment webhook integration
- ☐ Email notifications
- ☐ GET /api/photographer/events

Week 5-6: Admin & Photo Upload

- ☐ GET /api/admin/events/pending
- ☐ POST /api/admin/events/:id/mark-ready
- ☐ POST /api/events/:id/upload-photos
- ☐ Photo management (delete, organize)
- ☐ Gallery generation

Week 7: Public APIs

- ☐ GET /api/public/event/:slug
- ☐ GET /api/public/event/:slug/gallery
- ☐ POST /api/analytics/track
- ☐ Analytics aggregation

Week 8: Security & Deployment

- ☐ Encryption for sensitive data
- ☐ Security headers (CORS, HSTS, CSP)
- ☐ Rate limiting testing

- [] Database backups
 - [] Error logging
 - [] Deployment to Railway/Render
-

8. ENVIRONMENT VARIABLES

```
# Database
DATABASE_URL=postgresql://user:pass@neon.tech/db

# JWT
JWT_SECRET=min_32_char_random_secret
JWT_EXPIRY=24h
JWT_REFRESH_SECRET=another_secret
JWT_REFRESH_EXPIRY=7d

# Cloudinary
CLOUDINARY_CLOUD_NAME=your_name
CLOUDINARY_API_KEY=key
CLOUDINARY_API_SECRET=secret

# Email
EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=app_password

# Encryption
ENCRYPTION_KEY=32_char_hex_string

# Payment
RAZORPAY_KEY_ID=key
```

```
RAZORPAY_KEY_SECRET=secret

# CORS
FRONTEND_URL=http://localhost:3000 (dev) /
https://photomemories.ai (prod)

# Admin
ADMIN_EMAIL=admin@photomemories.ai

# Node
NODE_ENV=development
PORT=3001
```

9. DEPLOYMENT CHECKLIST

Before going live:

- ☐ All environment variables set
- ☐ Database migrations run
- ☐ HTTPS enforced
- ☐ CORS configured for production
- ☐ Rate limiting active
- ☐ Error logging setup
- ☐ Backups automated
- ☐ Encryption keys secured
- ☐ Payment integration tested
- ☐ Email service tested
- ☐ Subdomain routing tested

10. INTEGRATION WITH FRONTEND

Frontend needs from you:

1. Working APIs (all endpoints above)
2. Consistent error format

```
{  
  "success": false,  
  "error": "Error message",  
  "code": "ERROR_CODE"  
}
```

3. Proper HTTP status codes (200, 201, 400, 401, 403, 404)
4. CORS enabled
5. Token sent via httpOnly cookie (not header)

You need from frontend:

1. Token in requests (browser sends automatically via cookie)
2. Correct request body format
3. Calls to /api/analytics/track on user actions

11. IMPORTANT NOTES

Architecture

- **Frontend:** Deployed on Vercel (one codebase, multiple subdomains)
- **Backend:** Deployed on Railway/Render (separate from frontend)
- **Communication:** HTTPS only, Express middleware
- **Database:** Neon PostgreSQL
- **Storage:** Cloudinary
- **Security:** Encryption at rest, JWT tokens, rate limiting

Responsibility

-  You handle all backend logic
-  You manage database
-  You secure communication
-  You don't do frontend design
-  You don't do Figma design (Krish does)

Workflow (Remember)

1. Photographer submits request (PAID)
2. You create APIs for this
3. Admin (Krish) manually designs pages
4. You store HTML in database
5. Photographer uploads photos
6. Gallery auto-generates

7. Guest accesses via subdomain

You're ready to build. Start with DB schema & auth.

Good luck! 🚀